

Practical Session 6 - Approximation Algorithms (Solutions)

Problem 1: Knapsack Problem

(i) Alg1 non garantisce un'approssimazione costante

Testo: Dimostrare che l'algoritmo greedy che inserisce gli oggetti per densità decrescente (finché c'è spazio) non garantisce alcun fattore di approssimazione costante k .

Soluzione: Costruiamo un'istanza con capacità B e due oggetti: * Oggetto 1: $s_1 = 1, v_1 = 2$ (Densità: 2) * Oggetto 2: $s_2 = B, v_2 = B$ (Densità: 1)

L'algoritmo Alg1, ordinando per densità, valuterà prima l'Oggetto 1 e lo inserirà nello zaino. A questo punto, lo spazio rimanente sarà $B - 1$. L'Oggetto 2 non potrà entrare, quindi l'algoritmo terminerà con valore $v_{Alg1} = 2$. La soluzione ottima, invece, prenderà l'Oggetto 2 ottenendo un valore $v_{OPT} = B$. Il rapporto di approssimazione è $\frac{v_{OPT}}{v_{Alg1}} = \frac{B}{2}$. Dato che B può essere arbitrariamente grande, il rapporto non è limitato da alcuna costante k .

(ii) Ottimalità dello Zaino Frazionario (Fractional Knapsack)

Testo: Dimostrare che l'algoritmo greedy che prende gli oggetti interi $1, \dots, i - 1$ e una frazione dell'oggetto i per riempire esattamente B è ottimo.

Soluzione: Procediamo per assurdo usando l'**Argomento di Scambio (Exchange Argument)**. Supponiamo esista una soluzione ottima OPT diversa da quella generata dall'algoritmo Greedy (G). Poiché G seleziona le porzioni di oggetti rigorosamente in ordine di densità decrescente $\frac{v}{s}$, se $OPT \neq G$, allora OPT deve aver escluso una certa porzione di volume Δ di un oggetto con densità maggiore (che G ha preso) per includere un pari volume Δ di un oggetto con densità minore. Se in OPT scambiamo il volume Δ a densità minore con il volume Δ a densità maggiore, il volume totale rimane $\leq B$, ma il valore totale aumenta strettamente. Questo contraddice l'ipotesi che OPT fosse il valore massimo possibile. Dunque l'algoritmo Greedy produce la soluzione ottima.

(iii) Alg2 è una 2-approssimazione

Testo: L'algoritmo Alg2 prende i primi $i - 1$ oggetti. Se $v_i > \sum_{j=1}^{i-1} v_j$, allora restituisce solo l'oggetto i . Altrimenti restituisce $a_1, \dots, a_{i-1}$. Dimostrare che è una 2-approssimazione.

Soluzione: Utilizziamo il rilassamento del problema. Sappiamo che la soluzione ottima intera OPT è sempre minore o uguale alla soluzione ottima frazionaria OPT_{frac} :

$$OPT \leq OPT_{frac}$$

Sappiamo dal punto (ii) che il valore della soluzione frazionaria è dato dalla somma dei primi $i - 1$ oggetti più una *frazione* dell'oggetto i . Pertanto, essa è strettamente minore della somma dei primi i oggetti interi:

$$OPT \leq OPT_{frac} < \sum_{j=1}^{i-1} v_j + v_i$$

Siano $V_A = \sum_{j=1}^{i-1} v_j$ e $V_B = v_i$. L'algoritmo Alg2 restituisce il massimo tra V_A e V_B . Per una proprietà aritmetica fondamentale, il massimo tra due valori è sempre maggiore o uguale alla loro media:

$$ALG2 = \max(V_A, V_B) \geq \frac{V_A + V_B}{2}$$

Sostituendo l'upper bound trovato precedentemente, otteniamo:

$$ALG2 \geq \frac{V_A + V_B}{2} > \frac{OPT}{2}$$

Questo dimostra che Alg2 restituisce sempre un valore maggiore della metà dell'ottimo, garantendo così una 2-approssimazione.

Problem 2: Sonet Ring Loading Problem

Testo: Dato un anello di n nodi e un set di chiamate C , instradare ogni chiamata in senso orario o antiorario per minimizzare il carico massimo sui link. Fornire un algoritmo 2-approssimato.

1. Algoritmo (L'approccio del taglio) L'algoritmo rimuove la natura decisionale del problema "tagliando" iterativamente ogni arco dell'anello, trasformandolo in una linea retta dove l'instradamento è forzato.

- **Inizializza** il tracciamento della soluzione migliore a $+\infty$.
- **Per ogni** arco e da 0 a $n - 1$:
 - Rimuovi temporaneamente l'arco e dall'anello.
 - Inizializza a 0 i carichi di tutti gli altri archi.
 - **Per ogni** chiamata (i, j) in C :
 - Instrada la chiamata sull'unico percorso disponibile tra i e j .
 - Incrementa di 1 il carico di ogni arco attraversato.
- Calcola il carico massimo L_{max} tra tutti gli archi in questa configurazione.
- Se L_{max} è minore del minimo trovato finora, salva questo instradamento come il migliore.
- Ripristina l'arco e .
- **Ritorna** l'instradamento migliore.

2. Dimostrazione (2-Approssimazione) Sia OPT il carico massimo nella soluzione ottima. Nessun arco nella soluzione ottima ha un carico superiore a OPT .

- Scegliamo l'arco e^* che, nella soluzione ottima, ha il carico minore. Sappiamo che il suo carico nell'ottimo è $L_{e^*} \leq OPT$.
- Consideriamo la specifica iterazione del nostro algoritmo in cui decide di tagliare proprio l'arco e^* . Dividiamo le chiamate in due gruppi:
- **Gruppo A:** Chiamate che nell'ottimo non passavano per e^* . Il nostro algoritmo le instrada esattamente come l'ottimo. Il loro contributo al carico su qualsiasi arco è quindi $\leq OPT$.
- **Gruppo B:** Chiamate che nell'ottimo passavano per e^* . Il nostro algoritmo è costretto a deviarle. Il numero di queste chiamate è esattamente $L_{e^*} \leq OPT$. Nel peggiore dei casi, ognuna di esse aggiunge un carico pari a 1 a un altro arco. Il carico extra generato è quindi $\leq OPT$.
- Il carico massimo generato nell'iterazione in cui tagliamo e^* sarà quindi la somma dei due contributi:

$$\text{Carico}(e^*) = \text{Carico}_A + \text{Carico}_B \leq OPT + OPT = 2 \cdot OPT$$

- Poiché il nostro algoritmo sceglie l'instradamento globale con il carico massimo minimo tra tutti gli n possibili tagli, il risultato finale ALG sarà minore o uguale a quello ottenuto tagliando e^* :

$$ALG \leq \text{Carico}(e^*) \leq 2 \cdot OPT$$

Questo garantisce la 2-approssimazione.
